Android tablet app: developer guide

Written for: the Android developer building our tablet app. It covers the whole system, how a tablet is set up once, how staff log in with their passcode, and every use case of the **table screen** in every kind of restaurant, with the API call for each step.

Everything here is live and covered by tests. The full machine-readable list is `openapi.json` (OpenAPI 3), sent with this guide.

Part A. The system in one page

A1. Who uses what

Who	Where	How they get in
Our team (installers, support)	`https://admin.symularai.com` (web)	Email + password + phone code
Restaurant owner / manager	`https://admin.symularai.com` (web, same login)	Email + password
Tablets (table, POS, waiter, kitchen, kiosk, board)	**This app** → `https://app.symularai.com`	Set up **once** with a screen code; staff then use a **passcode (PIN)**
Guests	The table tablet	Nothing: they just tap

A2. The screen types

One app serves every tablet. The **screen code decides what the tablet becomes**: the app never asks "which screen is this?".

`device.type`	Screen	Built how	Staff passcode?
`table`	**Table screen** (guests order at the table)	**Native** (Part D)	No (optional, for staff actions)
`cashier`	POS (counter till)	Our web screen in a WebView	**Yes**, every shift
`waiter`	Waiter app (phone or small tablet)	Web screen in a WebView	**Yes**, every shift
`kds`	Kitchen display	Web screen in a WebView	No
`kiosk`	Self-order kiosk (coming soon)	Web screen in a WebView	No
`status`	Order-number board on a TV (coming soon)	Web screen in a WebView	No

Web screens are already finished and tested. Wrapping them saves months; the app adds what a browser can't do (start on boot, lock the tablet, keep the key safe, print by USB/Bluetooth).

A3. Network: Cloud or Hybrid

```
Cloud restaurant:   tablet --internet--> https://app.symularai.com
Hybrid restaurant:  tablet --restaurant Wi-Fi--> hub (small PC, http://192.168.x.x:8000) --> cloud
                    (if the hub is down, the tablet talks to the cloud directly)
```

The server tells the app which one it is (`network` in `/v1/devices/me`, §D9). Web screens already switch by themselves; the native table screen must do it (§D9).

A4. Conventions

- **Base URL:** `https://app.symularai.com` (production). Each restaurant has a short **code**, e.g. `demo`.
- **JSON** over HTTPS. Times are ISO-8601 UTC.
- **Money:** integer **LKR cents**. `95000` = Rs 950.00. The app never sends prices.

- **Names:** `{"en": "...", "si": "...", "ta": "..."}`. Show the guest's language; fall back to `en`.
- **Errors:** always `{"error": {"code": "...", "message": "..."}}` with an HTTP status. Show `message` (it is written for people); branch on `code`. Validation errors are 422 `validation_error`.
- **Headers:**
  - `X-Device-Token: <device_token>` on every call after setup.
  - `Authorization: Bearer <staff_token>` as well, while a staff member is logged in.
  - `Idempotency-Key: <uuid>` on creating orders and on payments (§D5).

Part B. Setup: once, for life

B1. What the installer does (in the Setup console)

1. Adds the tablet as a screen: type (table / POS / waiter / kitchen), name, and for a table its **table number** (e.g. `T5`), for a kitchen its **station** (e.g. `grill`).
2. The console shows:
   - **Restaurant code:** `demo`
   - **Screen code:** `K7MQ-4XPA-2R` (10 characters from `23456789ABCDEFGHJKMNPQRSTUVWXYZ`)
   - A **QR code** with both: `https://app.symularai.com/demo?code=K7MQ-4XPA-2R`

B2. What the app does on first start

Setup screen: "Scan the QR code" (camera) **or** type the restaurant code + screen code. Typing is forgiving: lower case and dashes don't matter.

```
POST /v1/devices/sign-in
{"venue": "demo", "code": "K7MQ4XPA2R",
 "client": {"install_id": "<UUID made once per install>", "app": "android", "app_version": "1.0.0",
            "model": "SM-X205", "os": "Android 14", "screen": "1920x1200"}}

200 {"device_token": "...",
     "device": {"id": "...", "type": "table", "name": "Table T5", "venue_id": "...",
                "venue_code": "DEMO", "station": null, "table_ref": "T5"},
     "replaced_another_device": false}
```

Error	Status	Show
`bad_screen_code`	400	"That code isn't right for this restaurant. Check it in the Setup console."
`screen_turned_off`	403	"This screen is turned off in the Setup console."
`too_many_attempts`	429	"Too many tries. Wait 5 minutes."
`cloud_unreachable`	503	(hub only) "Setting up needs the internet once."
network error	-	"No connection. Check the Wi-Fi." Retry.

Then:

1. **Save** `device_token` and `device` in Android **Keystore-encrypted** storage (EncryptedSharedPreferences). The token **never expires**.
2. Open the screen for `device.type` (B4).

`replaced_another_device: true` means this code was used on another tablet before; that tablet is now signed out. This is how a broken tablet is replaced: same code, new tablet.

B3. Every start after (boot, crash, update, power cut)

```
saved token? -- no --> Setup screen (B2)
     | yes
     v
GET /v1/devices/me -- 401 device_not_set_up --> delete the token > Setup screen
     | 200, or any other error / no network
     v
Open the screen for device.type (use the saved `device` if the server can't be reached)
```

The golden rule: only **HTTP 401 with** `error.code = "device_not_set_up"` (or WebSocket close **4401**) means "forget the setup". It happens when the installer:

- signed the same screen in on another tablet,
- made a new screen code for it, or
- turned the screen off.

Every other error (timeouts, 500, 409, no Wi-Fi) → keep the token, show the problem, retry. Never send the tablet back to setup because the internet dropped.

B4. Opening the right screen

`device.type`	**Open**
`table`	The native table screen (Part D)
anything else	WebView, full screen, at `https://app.symularai.com/#move=<handoff>`

`<handoff>` = `encodeURIComponent(base64(JSON))` of:

```
{"device_token": "<token>", "device": { ...the device object from sign-in... }, "slot": "main"}
```

The web app stores it and opens the right screen (`/demo/pos`, `/demo/waiter`, `/demo/kitchen` …). Do this on every start, so the WebView always has the current key.

WebView settings:

- JavaScript on, DOM storage on, keep cookies and storage.
- Allow cleartext HTTP to private addresses only (`192.168.*`, `10.*`), for the hub, with a `networkSecurityConfig`.
- Inject a native bridge for printing and device info (C4).
- If the page reports `device_not_set_up` (it shows its own sign-in page), the app may also check `/v1/devices/me` and go to its native Setup screen.

B5. Heartbeat

Every **60 s**: `POST /v1/devices/heartbeat` `{"client": {…same as sign-in…}}` → `{"ok": true, "server_time"}`. The Setup console uses it to show "online · Android 14 · SM-X205". (Web screens send it themselves; the native table screen must.)

B6. Kiosk behaviour (all types)

- Start on boot; lock task mode (screen pinning or an MDM); keep the screen on.
- **Hidden settings:** hold the top-left corner for 3 seconds → ask for a **manager passcode** (C2) → show: screen name, restaurant, server, app version, "Test printer", "Set up again".
- "Set up again" deletes the token and shows the Setup screen.

Part C. Staff passcode (PIN) login

The tablet is set up once. **People** log in on it with a 4-8 digit passcode each time they start using it (POS and waiter screens: at the start of a shift; table screen: only for a staff action).

C1. The flow

```
GET /v1/staff                      → [{"id","name","role","is_active"}]   (active staff, by name)
  show names as large buttons (no photos needed) → staff taps their name
  show a number pad → staff types the PIN
POST /v1/staff/pin-login {"staff_id": "...", "pin": "1234"}
  → 200 {"staff_token": "...", "token_type": "bearer", "staff": {"id","name","role","is_active"}}
```

Then send `Authorization: Bearer <staff_token>` with every call, **on this tablet only** (the token is tied to the tablet; on another one it gets 401).

Error	Status	Show
`unauthorized` "Wrong staff or PIN"	401	Shake the dots, clear the PIN
`pin_locked`	423	"Too many wrong PINs. Try again in 5 minutes." (5 wrong in a row locks that person for 5 minutes; a manager can also reset their PIN)

C2. Rules

Rule	Value
Session length	8 hours, then 401 "Session expired" → show the PIN pad again
Log out / switch user	Just forget the `staff_token` (no server call) and show the PIN pad
Table screen	Drop the staff session after **2 minutes** idle
POS / waiter	Keep it for the shift; "Lock" button returns to the PIN pad
Who may log in	Any active staff member; each **action** checks the role (C3)
Change PIN	Only the owner/manager, in the Owner app (not on the tablet)
Hybrid switch	A PIN session belongs to the server that made it; after switching hub ↔ cloud the PIN may be asked again

C3. What each role may do

Action	cashier	waiter	kitchen	manager / owner
Take orders for a table, open/close/clean tables, print, fire held items	Yes	Yes	Yes	Yes
Take cash / card payments, hold a bill while paying, mark served	Yes	Yes	-	Yes
Open / close a cash shift	Yes	Yes	-	Yes
Void, refund, release someone else's payment hold	-	-	-	Yes

The kitchen screen needs no PIN at all.

C4. Native bridge (for the WebView screens)

Inject `window.KioskNative` before the page loads:

```
window.KioskNative = {
  deviceInfo(): string,          // JSON: {"install_id","app":"android","app_version","model","os","screen"}
  printEscPos(base64, printer),  // send ESC/POS bytes to the printer; Promise, rejects on failure
};
```

C5. Printing on a printer plugged into the tablet

If the installer added a printer "plugged into this screen" (USB or Bluetooth), the tablet prints its slips. The app does this **natively for every screen type**, even while a WebView is open.

```
WS event  print.job {"job_id","label"}          (on the tablet's own channel)
GET  /v1/print/printers                         → [{"id","name","roles","paper_mm"}]
GET  /v1/print/jobs                             → waiting jobs (oldest first, max 20): catch-up after offline
GET  /v1/print/jobs/{id}                        → {"id","label","escpos_b64","text","printer":{...}}  (marks it
"sent")
POST /v1/print/jobs/{id}/result {"ok": true}    or {"ok": false, "error": "Paper out"}
```

- `escpos_b64` is ready to send: base64 of ESC/POS bytes, with the paper cut and, for cash receipts, the cash-drawer kick. Don't change it.
- **On start and on every reconnect:** `GET /v1/print/jobs` and print anything waiting. A printer that was off prints what it missed.
- **Failure:** report `ok: false` with a short reason; staff screens get a "print failed" alert.
- `device.command` `connect_printer`**:** show "Connect printer" (pick USB / Bluetooth printer) and remember the choice.

Part D. The table screen (native): every use case

D1. The restaurant's settings decide the flow

Read them from `GET /v1/devices/me` on start and again on the `venue.settings` event:

```
{
  "device":  {"id","type":"table","name":"Table T5","station":null,"table_ref":"T5"},
  "venue":   {"id","name","code","timezone","languages":["en","si","ta"],
              "release_rule":"A","dine_in_mode":"tab","serve_mode":"waiter",
              "attract_images":["/media/..."],
              "brand":{"color":"#f5a524","logo_url":"/media/...","theme":"light"},
              "board_page_seconds":8},
  "table":   {"ref":"T5","access":"staff_opens","open":false},
  "payments":{"cash":true,"lankaqr":true,"card":null},
  "modules": ["R-CORE","R-TABLE","R-LOYAL", ...] ,
  "network": {"mode":"cloud","hub_url":null,"cloud_url":"https://app.symularai.com","served_by":"cloud"},
  "staff":   null
}
```

Setting	Values	What changes on the table
`venue.dine_in_mode`	`tab`	**Pay after eating.** Orders go to the kitchen at once; more rounds add to the same bill; main pay button "Ask for the bill"
	`pay_first`	**Pay before cooking.** After "Place order" the guest pays (QR / card reader / at the counter); the kitchen starts after payment
`table.access`	`open`	Guests can order any time
	`staff_opens`	Shows **Table closed** until a waiter opens the table (`table.state` event). "Call staff" still works
`venue.serve_mode`	`waiter`	When food is ready: "On its way" (a waiter brings it)
	`self_pickup`	When ready: "Ready! Collect at the counter · order T005"
`payments.cash` / `lankaqr`	true/false	Show or hide that way to pay
`payments.card`	`"reader"`	A card reader is attached to the tablet: guests can tap alone
	`"softpos"`	Tap-on-tablet, **only while a staff member is logged in**
	`null`	Card is paid on the waiter's bank machine
`modules`	lacks `R-LOYAL`	Hide "rewards / phone number"
	lacks `R-SPLIT`	Hide "split the bill"
	lacks `R-PAY`	Hide QR and card-reader payments
`venue.languages`	`en`,`si`,`ta`	Language buttons on the welcome screen
`venue.brand`	colour, logo, light/dark	The whole screen wears the restaurant's colour and logo. No logo: show the name in a clean mark (no letter avatars)

The four common restaurant types:

Restaurant	dine_in_mode	access	serve_mode	Typical flow
Sit-down restaurant	`tab`	`staff_opens`	`waiter`	Waiter seats guests and opens the table → guests order rounds → food brought → ask for bill → pay → rate
Casual café	`tab`	`open`	`waiter`	Same, but guests can order as soon as they sit
Food court / quick service	`pay_first`	`open`	`self_pickup`	Order → pay by QR/card → cooking → "collect at counter"
Hotel / bar	`pay_first`	`staff_opens`	`waiter`	Waiter opens → order → pay → food brought

D2. Screens to build

Screen	When
Setup	No token, or after `device_not_set_up` (Part B)
Welcome	Idle, between guests. Logo, colour, `attract_images` slideshow, "Tap to order", languages
Table closed	`table.access = staff_opens` and `table.open = false`. Welcome + only **Call staff**
Menu	Category tabs, dish cards (photo, name, price, "+"); dishes without a photo as clean text cards
Dish sheet	Options (modifier groups), combo choices, notes, quantity
Cart	Lines, –/+, server-priced totals, Send / Place order
Order status	Placed → Cooking (ready in N min) → On its way / Ready to collect → Enjoy your meal
Bill / Pay	Ask for the bill (tab) or pay now (pay first)
Rate	Stars, tags, comment
Call staff	Always-visible button
Staff PIN	From hidden settings or a staff action (Part C)

D3. Menu

`GET /v1/menu` →

```
{"venue_id","at","categories":[{"id","name":{...},"items":[{
   "id","name":{...},"description":{...},"price":95000,"image_url":"/media/..."|null,
   "allergens":["nuts"],"is_available":true,
   "modifier_groups":[{"id","name":{...},"min_select":0,"max_select":1,
                       "modifiers":[{"id","name":{...},"price_delta":15000}]}],
   "combo_slots":[{"name":{...},"item_ids":["..."],"upcharges":{"<item_id>":10000}}],
   "suggest_ids":["..."],
   "offer":{"price":85500,"discount_bp":1000,"ends_at":"..."}|null}]}],
 "combo_parts":{ "<item_id>": {...item...} }}
```

- Only what is served **now** is returned (breakfast/lunch times are handled by the server).
- `is_available: false` → show "Sold out", block adding.
- `image_url` is relative: load `{server}{image_url}`.
- `min_select` / `max_select`: enforce per group (e.g. "Choose 1 size").
- `combo_slots`: one choice per slot (e.g. "Drink: Coke / Sprite (+Rs 100)").
- `suggest_ids`: "Goes well with…" in the dish sheet; lines added from there get `"source": "upsell"`.
- `offer`: show the offer price and the old price struck through.
- Reload on `menu.updated`, `menu.item_availability`, `menu.offers`, and `resync`.

D4. Cart and prices

Cart line:

```
{"menu_item_id":"...","quantity":2,"modifier_ids":["..."],"notes":"no onion","combo_choices":[],"source":null}
```

On every change: `POST /v1/orders/quote {"items":[lines]}` → `{"subtotal","discount_total","tax_lines":[{"code","name","rate_bp","amount"}],"tax_total","total"}`. Show these numbers, never your own sums.

Errors (400): `item_unavailable` (just sold out: mark it), `item_not_served_now`, `bad_modifiers`, `bad_combo`, `not_enough_stock`, `empty_cart`.

D5. Sending the order

```
POST /v1/orders            Idempotency-Key: <new UUID per tap of "Send">
{"items":[lines], "notes": null}
→ 201 order
```

(The table's number is added by the server; don't send `table_ref`.)

Never send an order twice: make the key when the guest taps Send; if the call fails (timeout, no network) **retry with the same key** until you get an answer. The server returns the first result for a repeated key.

Order fields used by the table: `id`, `token` (e.g. "T005"), `status`, `payment_status`, `is_tab`, `served_at`, `promised_at`, `items[{name,quantity,line_total,kds_status}]`, `subtotal`, `tax_lines`, `total`, `paid_total`, `payment_hold`.

Error	Status	Show
`table_closed`	409	"This table isn't open yet. We've called a waiter." (and send `call_waiter`)
`table_locked`	409	"Order T005 is waiting for payment. Please pay it first."
`bill_being_paid`	409	"Your bill is being paid. Please wait a moment."
price errors	400	As in D4

Tab (pay after eating): every Send adds a **round to the same order** (`is_tab: true`, event `order.round_added`). Keep showing one bill.

Change or cancel (only before the kitchen starts and before payment):

- `PUT /v1/orders/{id}/items {"items":[…]}` → 409 `order_locked` once cooking started.
- `POST /v1/orders/{id}/cancel {"reason":"Guest changed mind"}` → 409 `order_paid` / `bad_transition`.

D6. Order status (what the guest sees)

Order	Show
`status = placed`, not paid (pay first)	"Waiting for payment" + pay options
`status = in_kitchen`	**Cooking** · "ready in about N min" (from `promised_at`)
`status = ready`, `served_at = null`, serve_mode `waiter`	**On its way**
`status = ready`, serve_mode `self_pickup`	**Ready! Collect at the counter** · big token "T005" + chime
`served_at` set	**Enjoy your meal**
`cancelled` / `expired` / `voided`	"This order was cancelled" + Call staff

Refresh with `GET /v1/orders/{id}`, and live with the events in D9.

D7. Paying

A. Pay after eating (`tab`)

1. Guest taps **Ask for the bill** and chooses **Cash**, **Card** or **QR**: `POST /v1/service-requests {"kind":"request_bill","order_id":"…","pay_with":"cash"}` (choosing again changes `pay_with` without paging twice).
2. Show "Your waiter is bringing the bill" → when the request has `coming`, "Priya is coming".
3. The waiter brings the bill (and the card machine, or takes the cash and brings the change and receipt). The table needs no action.
4. **QR on the table instead:** `POST /v1/orders/{id}/payments/lankaqr` → `{"payment":{"id","qr_payload","amount","status":"pending"},"order"}` → draw `qr_payload` as a QR code; wait for `order.paid`. Close it with `POST /v1/orders/{id}/payments/{payment_id}/cancel`.

5. On `order.paid`: "Thank you!" → Rate (D8) → Welcome.

B. Pay first (`pay_first`) After **Place order** show the ways that are on:

- **QR:** as A4.
- **Card reader** (`payments.card = "reader"`): `POST /v1/orders/{id}/payments/card-terminal` + `Idempotency-Key` → payment `pending` ("Tap or insert your card on the reader") → `order.paid` or `payment.failed`. Cancel with `/payments/{payment_id}/cancel`.
- **Pay at the counter:** show the token big: "Pay at the counter · T005".

After payment the kitchen starts: status → Cooking.

Split bill (only if `modules` has `R-SPLIT`): send `"amount"` (cents) for one share in the QR/card call; event `payment.partial` until fully paid.

Error	Status	Show
`being_paid`	409	"{name} is taking the payment" (a waiter is paying it right now)
`already_paid`	409	"This bill is paid. Thank you!"
`lankaqr_unavailable`	503	"QR payment isn't available right now. Your waiter will help."
`payment_method_off`	400	Hide that way
`no_card_terminal`, `softpos_needs_staff`	400	"Card isn't available on this screen. Your waiter will help."
`module_not_in_plan`	403	Hide that feature

D8. Calling staff, feedback, the end of the visit

Action	Call
Call the waiter	`POST /v1/service-requests {"kind":"call_waiter"}` → 201 request. Pressing twice doesn't page twice
See my open calls	`GET /v1/service-requests?mine=true` → `[{"id","kind","status":"open|acknowledged|done","pay_with","coming":{"staff_id","name"}|null}]`
Rate the meal	`POST /v1/feedback {"rating":1-5,"tags":["food","service"],"comment":"…","order_id":"…"}` → 201. One per order (sending again updates it)

- Show "Priya is coming" when `coming` is set; clear when `done`.
- Feedback tags: `food`, `service`, `speed`, `value`, `clean`, `slow`, `cold`, `wrong_order`. 1-2 stars alert the staff, and a waiter comes to the table.
- Ask for the rating when the food is served (`served_at`) and again after paying; always optional.
- **End of visit:** on `table.cleared` (waiter closed the table) or a paid tab → forget the order, the cart and any phone number → Welcome. The waiter's app shows "Needs cleaning" until they tap Cleaned.
- **Privacy reset:** 30 s idle with nothing open → Welcome. Nothing of the last guests stays on screen.

D9. Live updates and the network

WebSocket: `wss://app.symularai.com/v1/ws?token=<device_token>` (on a hub: `ws://<hub>/v1/ws?token=…`)

- First message `{"type":"hello","epoch","seq"}`. Remember `epoch` and the last `seq`.
- Send `{"type":"ping"}` every 25 s (reply `pong`).
- Reconnect with back-off 1 s → 15 s, adding `&since=<seq>&epoch=<epoch>`: missed events are replayed, or you get `{"type":"resync"}` → reload `/devices/me`, the menu and your order.
- Events arrive as `{"seq","epoch","type","data","at"}`.
- Close **4401** → setup is gone (as `device_not_set_up`). Close **4409** → use the hub.

Event	data	Table screen does
order.created, order.round_added, order.released, order.ready, order.served, order.paid, order.payment_hold, order.completed, order.cancelled, order.expired, order.voided, payment.partial, payment.failed, payment.cancelled, kds.*	{"order", …}	If order.id is ours: update status. Chime on order.ready
service.acknowledged, service.done	{"request"}	If request.device_id is ours: "Priya is coming" / clear
table.state	{"table_ref","open"}	Our table: show the menu, or Table closed
table.cleared	{"table_ref"}	Our table: back to Welcome, forget everything
menu.updated, menu.item_availability, menu.offers		Reload the menu
venue.settings	{"settings"}	Reload /v1/devices/me (brand, payments, modes)
device.command	{"command":"reload"|"identify"|"connect_printer"}	Reload / show the table name big for 8 s / C5
print.job	{"job_id","label"}	C5

Hybrid restaurants (network.mode = "hybrid"):

1. Talk to hub_url first; if it can't be reached, use cloud_url.
2. The cloud answers **409** use_hub (WebSocket close **4409**) while the hub is alive → switch to the hub.
3. While on the cloud, call GET {hub_url}/v1/health every 10 s; switch back when it answers.
4. If the hub stops answering but the cloud still says use_hub, its address changed: read /v1/devices/me from the cloud again for the new hub_url.
5. The same device token works on both.

No connection at all: keep the cart; show "No connection. Your order is safe and will be sent when we're back"; retry Send with the **same** Idempotency-Key. Call staff: retry quietly.

D10. Staff on the table tablet (optional)

From hidden settings or a "Staff" button, a waiter logs in with their PIN (Part C). While logged in:

- order at a closed table (the table opens automatically),
- "hold": true on a line (send later with POST /v1/orders/{id}/fire),
- take a SoftPOS card tap (payments.card = "softpos").

Drop the staff session after 2 minutes idle.

Part E. The other screens (inside the WebView)

These are finished web screens. The app only needs Part B (setup), C4/C5 (bridge, printing) and B6 (kiosk behaviour). For reference, this is what they do:

Screen	Staff passcode	Main calls
POS (`cashier`)	Yes	`POST /v1/shifts/open {"opening_float"}` → orders → `POST /v1/orders/{id}/payment-hold` → `/payments/cash {"tendered"}` or `/payments/card {"approval_code"}` (bank machine) → receipt prints → `POST /v1/shifts/{id}/close {"counted_cash"}`
Waiter (`waiter`)	Yes	`GET /v1/tables` (map with states: calling, bill, ready, cooking, eating, cleaning, free, closed) → open table, order for a table, answer calls (`/service-requests/{id}/ack`, `/done`), mark served, print and take the bill, `POST /v1/tables/{ref}/cleaned`, see feedback
Kitchen (`kds`)	No	`GET /v1/kds/tickets` → start / done per item, bump the order, recall, remake, sold out (`PUT /v1/menu/items/{id}/availability`)

Money safety built into the POS and waiter screens:

- **Payment hold:** while one person takes a payment, nobody else can (`being_paid`).
- **Idempotency keys:** a retried cash payment is never counted twice.
- **"Checking the payment…":** shown when the answer is lost, before trying again.

Part F. Rules the app must keep

1. Never send prices; show the server's totals.
2. `Idempotency-Key` on every `POST /v1/orders` and payment call; retry with the same key.
3. Only `401 device_not_set_up` / close 4401 signs the tablet out. Everything else: show, retry.
4. The device token lives in encrypted storage; never in logs, URLs shown to people, or analytics.
5. Call staff always works, even on the closed screen and offline (retry).
6. Back to Welcome after idle and after the visit; nothing personal stays.
7. Card taps on the tablet only while staff are logged in (SoftPOS rule).
8. All guest text in en / si / ta.
9. Print jobs: report every result (`ok: true/false`); catch up on start.

Part G. Testing

Against a local server (your computer):

```
uv sync && uv run kiosk initdb && uv run kiosk seed && uv run kiosk serve --host 0.0.0.0 --port 8000
```

1. Open `http://<computer IP>:8000/platform`, log in (demo login is filled in), open **Demo Café** → Setup console → **Screens** → add a table (T1) → note the screen code.
2. In the app: server `http://<IP>:8000`, restaurant `demo`, that code.
3. Act as staff in a browser: waiter `http://<IP>:8000/demo/waiter`, POS `/demo/pos`, kitchen `/demo/kitchen`. Demo PINs: Nimal **1111** (cashier), Priya **3333** (waiter).
4. Try each restaurant type from D1: in the Setup console → Restaurant, switch "Pay after eating / Pay first", "Staff open tables", "Waiter brings food / Guests collect".
5. Fake payments (test servers only): QR paid `POST /v1/dev/mock-gateway/{gateway_ref}/succeeded`; card tap `POST /v1/dev/mock-terminal/{ref}/succeeded`.

Done when:

- Set up once with the QR; after a reboot it opens by itself; signing the code in on a second tablet sends the first one back to setup.
- POS and waiter open in the WebView and ask for the staff PIN; 5 wrong PINs lock for 5 minutes.
- Table: closed screen → opens live when the waiter opens the table.
- Order → kitchen → Cooking → On its way / Ready to collect → Enjoy your meal.
- Ask for the bill (Cash) appears on the waiter's app; after payment the table resets.
- Pay-first with QR: kitchen starts only after payment.
- A rating appears in the Owner app; 1-2 stars alert the waiter.
- Wi-Fi off during Send → back on → the order arrives **once**.
- A USB printer on the POS prints receipts; unplugged → "print failed" → reprints later.
- Hybrid: turning the hub off and on switches to the cloud and back.